

FULL STACK DEVELOPMENT

FASE 4 | AULA 01 -

NOTIFICATIONS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

Nesta aula, exploraremos o conceito de notificações em sistemas e aplicações. Abordaremos os tipos de notificações, suas implementações e as melhores práticas para garantir uma comunicação eficiente com os usuários.



HANDS ON

Nesta aula, exploraremos notificações em aplicativos móveis com Firebase. Existem três tipos principais de notificações: push notifications (enviadas do servidor para o dispositivo), in-app notifications (aparecem durante o uso do aplicativo) e e-mail notifications (enviadas para o e-mail do usuário). Elas são importantes para engajamento, retenção e conversão de usuários.

Configuraremos o Firebase para push notifications criando um projeto no Firebase Console, adicionando o SDK ao projeto web, solicitando permissão ao usuário e configurando listeners para receber notificações. Utilizaremos a web notifications API para enviar notificações simples e em resposta a eventos e integraremos um backend com Node.js para enviar notificações condicionais. Boas práticas incluem garantir relevância, personalização, controle do usuário e timing adequado das notificações para evitar interrupções desnecessárias.

SAIBA MAIS

Push notifications são mensagens que podem ser enviadas para os dispositivos dos usuários mesmo quando eles não estão utilizando ativamente o aplicativo. Elas são uma ferramenta poderosa para manter os usuários engajados e informados sobre novidades, eventos ou atualizações importantes. No contexto de React, especialmente em React Native para aplicativos móveis, a implementação de push notifications pode ser feita utilizando bibliotecas específicas e serviços externos.

Para o funcionamento das push notifications, um serviço externo, como **Firestore Cloud Messaging (FCM)** ou Apple Push Notification Service (APNs), é utilizado.

Firestore Cloud Messaging (FCM)

Firestore Cloud Messaging (FCM) é um serviço gratuito oferecido pelo Google que permite enviar mensagens e notificações para aplicativos em várias plataformas, incluindo Android, iOS e web. As principais funcionalidades do FCM incluem envio de mensagens, que permite enviar mensagens de notificação e dados para seus aplicativos; segmentação de público, com a possibilidade de enviar mensagens para segmentos específicos de usuários; mensagens programadas, que permitem enviar mensagens em horários específicos; e notificações baseadas em condições, que possibilitam o envio de notificações com base em condições predefinidas.

Existem diferentes tipos de notificações no FCM, como push notifications, que são enviadas do servidor diretamente para o dispositivo do usuário, mesmo quando o aplicativo não está ativo; in-app notifications, que aparecem quando o usuário está utilizando o aplicativo; e e-mail notifications, enviadas para o endereço de e-mail do usuário para informá-lo sobre atualizações ou atividades importantes.

As notificações são importantes por diversos motivos, como engajamento, que ajuda a manter os usuários engajados e informados sobre atualizações importantes; aumento da taxa de retenção, ao lembrar os usuários de voltar ao aplicativo; e conversão, guiando os usuários para ações específicas, como completar uma compra ou participar de uma promoção. No entanto, é importante enfrentar desafios e seguir boas práticas, garantindo que as notificações sejam relevantes e valiosas para o

usuário, evitando o envio excessivo de notificações para não irritar os usuários, e personalizando as notificações de acordo com as preferências e comportamentos dos usuários.

Para que um dispositivo do usuário receba notificações, ele deve estar registrado, o que geralmente envolve obter permissão do usuário e registrar o dispositivo com o serviço de notificação. Quando o servidor deseja enviar uma notificação, ele deve seguir um processo que envolve a preparação da mensagem, a autenticação com o serviço de notificações (como FCM ou APNs) e o envio da mensagem para o serviço de notificações, que então a encaminha para o dispositivo do usuário.

Enviar notificações push envolve vários passos críticos que devem ser configurados corretamente para garantir que as notificações sejam entregues de maneira confiável e segura. A escolha entre FCM e APNs dependerá das plataformas de destino e das necessidades específicas do aplicativo. Além disso, é importante gerenciar adequadamente os tokens de dispositivos e lidar com eventuais erros de entrega para manter uma boa experiência de usuário.

Configuração do Firebase para push notifications

Passo 1: criar um projeto no Firebase

1. Acesse o Firebase Console.
2. Crie um novo projeto ou selecione um existente.
3. Navegue até a seção "Cloud Messaging" e anote as credenciais necessárias.
4. Crie um novo projeto ou selecione um existente.

Criar um novo projeto

1. Clique no botão "Add project" ou "Adicionar projeto".
2. Digite um nome para o seu projeto. Por exemplo, "MeuProjetoNotificacoes".
3. Você pode definir o Google Analytics para o seu projeto (opcional). Para simplificar, escolha "Continuar sem o Google Analytics" e clique em "Criar projeto".

4. Aguarde enquanto o Firebase cria seu projeto. Isso pode levar alguns segundos. Quando concluído, clique em "Continuar".

Selecionar um projeto existente

1. Na lista de projetos, clique no nome do projeto que você deseja usar.
2. Navegue até a seção "Cloud Messaging" e anote as credenciais necessárias.
3. No menu de navegação à esquerda, clique em "Project settings" (ícone de engrenagem) e selecione "Project settings".
4. Vá para a aba "Cloud Messaging" e você encontrará duas informações importantes: sender ID (um identificador único para o seu projeto e server key (a chave do servidor que será usada para autenticar suas solicitações de envio de notificações).

Passo 2: Adicionar Firebase ao seu projeto web

Para adicionar o SDK do Firebase ao seu projeto, siga estas etapas:

1. No menu de navegação à esquerda, clique em "Project settings" (ícone de engrenagem) e selecione "Project settings".
2. Na aba "General", na seção "Your apps", clique no ícone "</>" (Web) para configurar uma nova aplicação web.
3. Insira um nome para seu app web e clique em "Register app".
4. Siga as instruções para adicionar o Firebase SDK ao seu projeto. Isso geralmente envolve adicionar os scripts do Firebase ao seu arquivo HTML principal ou usar módulos ES no seu arquivo JavaScript.

```
<script src="https://www.gstatic.com/firebasejs/9.1.3/firebase-app.js"></script>
<script src="https://www.gstatic.com/firebasejs/9.1.3/firebase-
messaging.js"></script>
```

Para configurar o Firebase no seu código JavaScript, adicione o seguinte código ao seu arquivo JavaScript principal para inicializar o Firebase e configurar o Cloud Messaging:

```
import { initializeApp } from "firebase/app";
import { getMessaging, getToken, onMessage } from "firebase/messaging";

// Suas configurações do Firebase retiradas do console
const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_AUTH_DOMAIN",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_STORAGE_BUCKET",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_APP_ID",
};

// Inicializa o Firebase
const app = initializeApp(firebaseConfig);

// Inicializa o Cloud Messaging
const messaging = getMessaging(app);
```

Passo 3: solicitar permissão do usuário

Para que seu aplicativo possa enviar notificações push, você precisa solicitar permissão ao usuário. Para isso, adicione o seguinte código ao seu arquivo JavaScript principal para solicitar permissão ao usuário para exibir notificações:

```
const requestPermission = async () => {
  try {
    await Notification.requestPermission();
    console.log('Notification permission granted.');
```

// Obtém o token do dispositivo para enviar notificações

```
const token = await getToken(messaging, { vapidKey: 'YOUR_VAPID_KEY' });
console.log('FCM Token:', token);
  } catch (error) {
    console.error('Unable to get permission to notify.', error);
  }
};
```



```
// Chame a função para solicitar permissão  
requestPermission();
```

O código mostrado já inclui a lógica para obter o token do dispositivo após a permissão ser concedida. Esse token será usado para enviar notificações específicas para o dispositivo do usuário.

Passo 4: receber notificações no navegador

Uma vez que a permissão foi concedida e o token foi obtido, você pode configurar seu aplicativo para receber notificações.

Para configurar o listener para receber notificações, adicione o seguinte código ao seu arquivo JavaScript principal:

```
onMessage(messaging, (payload) => {  
  console.log('Message received. ', payload);  
  // Personalize a notificação aqui  
  new Notification(payload.notification.title, {  
    body: payload.notification.body,  
  });  
});
```

Esse código cria uma nova notificação utilizando a Web Notifications API. Você pode personalizar o conteúdo da notificação com base nos dados recebidos do payload.

Usando a Web Notifications API

Passo 1: entender a Web Notifications API

A Web Notifications API permite que os(as) desenvolvedores(as) enviem notificações que aparecem fora do contexto de uma página da web, oferecendo uma

maneira de manter os usuários informados mesmo quando não estão ativamente usando o site.

Passo 2: solicitar permissão e enviar uma notificação simples

Verifique se o navegador suporta a API e solicite permissão:

```
if ("Notification" in window) {  
  Notification.requestPermission().then((permission) => {  
    if (permission === "granted") {  
      new Notification("Hello, World!", {  
        body: "This is a simple notification",  
        icon: "icon.png",  
      });  
    }  
  });  
}
```

Passo 3: enviar notificações em resposta a eventos

Envie uma notificação em resposta a um evento do usuário, como um clique de botão:

```
document.getElementById("notifyButton").addEventListener("click", () => {  
  new Notification("Button Clicked", {  
    body: "You clicked the button!",  
    icon: "icon.png",  
  });  
});
```

Uma abordagem avançada é a integração com Backend para envio de notificações. Para isso, iremos configurar um backend (Node.js como exemplo).

Instale o Firebase Admin SDK:

```
npm install firebase-admin
```

Configure o Firebase Admin no seu servidor:

```
const admin = require("firebase-admin");
const serviceAccount = require("path/to/serviceAccountKey.json");

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount),
});

const message = {
  notification: {
    title: "New Notification",
    body: "This is a notification from the backend!",
  },
  token: "USER_DEVICE_TOKEN",
};

admin.messaging().send(message)
  .then((response) => {
    console.log("Successfully sent message:", response);
  })
  .catch((error) => {
    console.log("Error sending message:", error);
  });
```

Crie lógica no backend para enviar notificações baseadas em eventos específicos, como novas mensagens ou atualizações de dados.

Melhores práticas e considerações de UX

Garanta que as notificações sejam relevantes para o usuário e adicionem valor à experiência. É importante usar dados do usuário para personalizar as notificações, tornando-as mais impactantes. Também devemos permitir que os usuários escolham que tipos de notificações desejam receber e enviar notificações em momentos apropriados para evitar interrupções desnecessárias.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos o conceito de notificações e sua importância em sistemas e aplicações modernas. Exploramos os diferentes tipos de notificações, incluindo Push Notifications, In-app Notifications e Email Notifications.

Além disso, aprendemos a implementar notificações usando Firebase e a Web Notifications API e discutimos as melhores práticas para garantir uma boa experiência de usuário.

EXEMPLO

REFERÊNCIAS

McILREAVY, S. **Mastering Firebase for Android Development**: Build real-time mobile applications using Firebase and Android. [s.l.]: Packt Publishing, 2018.

RAY, M.; BLOCK, J. **Web Push Notifications**: Everything you need to know to get started. [s.l.]: O'Reilly Media, 2020.

RICHARDSON, A. **Firebase Essentials** - Android Edition. [s.l.]: Leanpub, 2017.

EXEMPLO

PALAVRAS-CHAVE

Palavras-chave: React Native. Front-end. Notification.

EMSE

POS TECH